

Programozási nyelvek 1

Szathmáry László
Debreceni Egyetem
Informatikai Kar

8. előadás

- fájlkezelés
- mutatók és tömbök, mutatók és sztringek
- nem módosítható és módosítható sztringek
- rendezés

(utolsó módosítás: 2026. febr. 3.)

2025-2026, 2. félév



Fájlkezelés

```
1  #include "prog1.h"
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <string.h>
5
6  #define MAX 1000
7
8  int main()
9  {
10     string fajlnev = "szoveg.txt";
11     char sor[MAX];
12
13     FILE *fp = fopen(fajlnev, "r");
14
15     if (fp == NULL)
16     {
17         printf("Hiba a %s file megnyitاسakor!\n", fajlnev);
18         exit(1);
19     }
20
21     while (fgets(sor, MAX, fp) != NULL)
22     {
23         sor[strlen(sor) - 1] = '\0';
24         printf("%s\n", sor);
25     }
26
27     fclose(fp);
28
29     return 0;
30 }
```

Szöveges fájl
olvasása soronként.

input fájl neve

hibakezelés

feldolgozás soronként

Fájl bezárása!

Fájlkezelés

```
1  #include "prog1.h"
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <string.h>
5  #include <math.h>
6
7  int main()
8  {
9      string fajlnev = "out.txt";
10
11     FILE *fp = fopen(fajlnev, "w");
12
13     if (fp == NULL)
14     {
15         printf("Hiba a %s fajl megnyitasakor!\n", fajlnev);
16         exit(1);
17     }
18
19     fprintf(fp, "Hello World!\n");
20     fprintf(fp, "2 + 2 = %d\n", 2 + 2);
21     fprintf(fp, "pí értéke: %lf\n", M_PI);
22     char c = 'a';
23     fprintf(fp, "az abece legelső betűje: %c\n", c);
24
25     fclose(fp);
26
27     return 0;
28 }
```

Szöveges fájlba
való írás.

output fájl neve

hibakezelés

A fájlba való írás
nagyon hasonló a
képernyőre való
íráshoz.

Fájl bezárása!

Fájlkezelés

Megnyitási módok:

- r read, olvasás
- w write, írás (ha létezik a fájl, akkor felülírja!)
- a append, hozzáfűzés (hozzáír a fájl végéhez, az eredeti tartalom megmarad)

Átírányítás

eszköz	azonosító	megnevezés	Alapesetben mihez van rendelve?
stdin	0	szabványos bemenet	billentyűzet
stdout	1	szabványos kimenet	képernyő
stderr	2	szabványos hibakimenet	képernyő

Minden egyes elindított folyamat esetén három alapértelmezett eszköz kerül hozzárendelésre a folyamathoz. Ezek:

- szabványos bemenet, ahonnan a beérkező adatokat olvassa
- szabványos kimenet, ahova a program ír
- szabványos hibakimenet, ahova a program a futása során fellépő hibákra adott hibaüzeneteit kiírja

Mind a bemenet, mind pedig a kimenet (a hibakimenet is) átirányítható egy tetszőleges állományba.

Átírányítás

< file	stdin átírányítása, a megadott állományból olvas
> file	stdout átírányítása, a megadott fájlba ír
>> file	stdout átírányítása, a megadott fájl végéhez hozzáír (append)
2> file	stderr átírányítása
&> file	stdout és stderr átírányítása
2>&1	a stderr-t ugyanoda irányítja, ahova az stdout irányítva lett
> /dev/null	stdout átírányítása a „szemetesbe”

A sizeof operátor

A sizeof unáris operátorral egy kifejezés, típus, változó, struktúra tárbán elfoglalt méretét lehet lekérdezni.

Az operátor által visszaadott érték byte-ban értendő.

```
sizeof(char) // 1, ui. a char típus mérete 1 byte
```

```
int x = 5;  
sizeof(x) // 4, ui. x int típusú, így 4 byte-ot foglal
```

```
double szamok[] = { 1.0, 2.0, 3.0 };  
sizeof(szamok) // 24, ui. 3 * 8 = 24
```

Mutatók és tömbök

```
int szamok[] = { 10, 20, 30 };
```

```
int *p = &szamok[0];      // p a 10-es elemre mutat
++p;                      // p a köv. elemre mutat (20)
```

```
p = &szamok[0];           // p a 10-es elemre mutat
*(p + 0)                  // a p által mutatott elem (10)
*(p + 1)                  // a p utáni első elem (20)
```

```
p[1]                      // ez a két jelölés
*(p + 1)                  // ekvivalens, ugyanazt jelentik
```

```
p = &szamok[0];           // ezek is
p = szamok;               // ekvivalensek
```

Amikor leírjuk egy tömb nevét, az valójában a 0. indexű elemének a címét jelenti!

Mutatók és sztringek

```
char* s = "hello";
```

```
char *p1 = s;           // a 'h' betűre mutat  
char *p2 = s + 4;       // az 'o' betűre mutat
```

```
printf("%c\n", s[0]);    // h  
printf("%c\n", s[1]);    // e
```

```
printf("A távolság: %d\n", p2 - p1);    // 4
```

Amikor leírunk egy sztringet literálként, vagy amikor karaktertömbként hivatkozunk rá, akkor ez valójában a 0. indexű elemének a címét jelenti.

Nem módosítható és módosítható sztringek

```
1  #include <stdio.h>
2
3  int main()
4  {
5      const char* s = "hello";
6
7      s[0] = 'H';
8
9      printf("%s\n", s);
10
11     return 0;
12 }
13
```

hiba

A "hello" karaktersorozat -- mivel literálként adtuk meg -- a memória egy nem módosítható területén lesz elhelyezve a program indulásakor. A `const` kulcsszó nélkül lefordulna, de a 7. sor miatt hibával terminálna a program. A `const` kulcsszónak köszönhetően már fordításkor hibát fogunk kapni.

Nem módosítható és módosítható sztringek

```
1  #include <stdio.h>
2
3  int main()
4  {
5      char text[] = "hello";
6      // char text[] = { 'h', 'e', 'l', 'l', 'o', '\0' };
7
8      text[0] = 'H';
9
10     printf("%s\n", text);
11
12     return 0;
13 }
14
```

OK

Itt egy karaktertömböt inicializálunk egy sztring karaktereivel. Az 5. és a 6. sor ekvivalens.

A text nevű karaktertömb így már módosítható.

Rendezés

Adatszerkezetek és algoritmusok órán számos rendezési algoritmust tanultak. Álljon itt egy rendezési algoritmus:

```
2
3 // egyszerű kiválasztásos rendezés (egyszerű, de lassú)
4 // helyben rendez
5 void tomb_rendez(int n, int tomb[])
6 {
7     for (int i = 0; i < n - 1; ++i)
8     {
9         for (int j = i + 1; j < n; ++j)
10        {
11            if (tomb[j] < tomb[i])
12            {
13                int temp = tomb[i];
14                tomb[i] = tomb[j];
15                tomb[j] = temp;
16            }
17        }
18    }
19 }
```

Házi feladat

- A K & R-féle „C Bibliában” nézzék át azokat a részeket, amikről szó volt az előadáson (**pl. 5. fejezet**).
- Juhász István jegyzetéből nézzék át azokat a fogalmakat, amikről szó volt az előadáson ([link](#)).

Szorgalmi

- *A Pirates of Silicon Valley* (magyarul: Számító emberek) c. film megtekintése (1999). [[IMDb link](#)]